

JavaScript Fundamentals — Notes

Console basics · Variables · Data Types · Operators · Strings · null/undefined (Apna College)

1. Using the Console

JS console uses **REPL** — Read, Evaluate, Print, Loop. It reads input, computes/calculates, prints the result, and loops back for more input.

Setup: VS Code files → `.html`, `.css`, `.js`

Clear console: `Cmd+K` (Mac) / `Ctrl+L` (Windows)

2. What is a Variable?

A variable is simply the **name of a storage location** in memory.

```
a = 10
age = 23
name = "Tony"
```

Memory box: `a→10`, `age→23`, `name→"Tony Stark"`. The variable name points to the value stored in memory.

3. Data Types in JS — Primitive Types

- Number
- Boolean
- String
- Undefined
- Null
- BigInt
- Symbol

Example (Instagram profile):

- Follow button → yes/no → `boolean`
- Followers count → `number`
- Name/bio text → `string`

4. Numbers in JS

- Positive (14) & Negative (-4)
- Integers (45, -50)
- Floating numbers — with decimal (4.6, -8.9)

Unlike C/C++/Java (which have separate `int` and `float` types), **JS has one unified Number type** for both integers and decimals.

Floating-point precision quirk: `0.9999999999999999` rounds to `1` in the console (limited decimal precision).

5. Operations in JS

```
a = 20, b = 10
sum = a + b      diff = a - b
prod = a * b      div = a / b
rem = a % b      // modulo (remainder)
```

Modulo: `12 % 5 = 2` (remainder). Used to check even/odd: `n % 2 === 0` → even.

Exponentiation (power): `2**3 = 8` → i.e. `a**b` = a multiplied by itself b times (e.g. $2^4 = 2 \times 2 \times 2 \times 2 = 16$)

Operands + operator: in `a + b`, `a` and `b` are **operands**, `+` is the **operator**.

6. NaN in JS

NaN = "Not-A-Number" — a special value representing an invalid/undefined numeric result.

```
0 / 0      // NaN
NaN - 1    // NaN
NaN * 1    // NaN
NaN + NaN  // NaN
```

Any operation involving NaN produces NaN. Interestingly, `typeof NaN` → `'number'`.

7. Operator Precedence (BODMAS)

General order of solving an expression, from highest to lowest priority:

1. `()` — Parentheses
2. `**` — Exponentiation
3. `*` / `%` — evaluated left → right
4. `+` / `-` — evaluated left → right

Example: `(5+2)/7 + 1*2` → `7/7 + 1*2` → `1 + 2` → `3`

Practice:

- `(2+1)*3` → `3*3` → `9`
- `3/1 + 2**2` → `3/1 + 4` → `3+4` → `7`
- `4 + 1*6/2` → `4 + 6/2` → `4+3` → `7`

8. let Keyword — Declaring Variables

```
let age = 23;
age = age + 1;      // 24

let cgpa;           // declared, no value
cgpa = 8.9;

let num1 = 1;
let num2 = 2;
let sum = num1 + num2; // 3
```

`let` allows re-assignment of values (e.g. `age` can change from `23` → `24` → `23`).

9. const Keyword

Values of constants **cannot be re-assigned** and **cannot be re-declared**.

```
const year = 2025;
year = 2026;      // Error!
year = year + 1;   // Error!

const pi = 3.14;
const g = 9.8;
```

Console proof: `const newNum = 5; newNum = 10;` → **Uncaught TypeError: Assignment to constant variable.**

`let area = pi * r ** 2` → **ReferenceError: r is not defined** if variable name doesn't match (used `rad` instead of `r`).

10. var Keyword

`var` is the **old syntax** of writing variables (predecessor to `let/const`).

```
var age = 23;
var cgpa = 8.9;
var num1 = 1;
var num2 = 2;
var sum = num1 + num2;
```

11. Assignment Operators

```
age = age + 1 → age += 1
age = age - 1 → age -= 1
age = age * 1 → age *= 1
```

Shorthand form: `a = b` means value of `b` assigned to `a`.

12. Unary Operators (Increment/Decrement)

Binary operators need 2 operands (e.g. `a+b`, `a*b`, `a/b`). **Unary operators** act on a single operand.

```
age++ // increment
age-- // decrement
```

Pre-increment ++a (Change, then use)

```
let age = 10;
let newAge = ++age;
// age = 11, newAge = 11
```

Post-increment a++ (Use, then change)

```
let age = 10;
let newAge = age++;
// age = 11, newAge = 10
```

Similarly: `--a` = pre-decrement, `a--` = post-decrement.

Practice:

```
let num = 5;
let newNum = num++; // newNum=5, num=6
newNum = ++num;     // num=7, newNum=7
```

13. Identifier Rules

All JS variables must be identified with **unique names** (identifiers).

- Names can contain letters (A-Z, a-z), digits (0-9), underscores, and dollar signs. **No spaces.**
- Names must begin with a letter.
- Names can also begin with `$` or `_`.
- Names are **case sensitive** (`y` and `Y` are different variables).
- Reserved words (JS keywords) CANNOT be used as names.

14. camelCase — Naming Conventions

- camelCase** — JS naming convention → `let fullName;`
- snake_case** → `let full_name;`
- PascalCase** → `let FullName;`

15. Boolean in JS

Boolean represents a truth value → **true or false** / yes or no.

```
let age = 23;
let isAdult = true;

let age = 13;
let isAdult = false;
```

JS is **dynamically typed** — a variable's type can change:
`let a = 5; a = true;` is valid (type changes freely).

16. What is TypeScript?

Statically typed language (unlike JS which is dynamically typed). Designed by Microsoft.

```
a = 5 // OK
a = true // ✗ Error in TS (type mismatch)
a = 10 // OK (still number)
```

17. Practice: Errors

a) `let lage = 5; let 2age = 10;` → Invalid — identifiers cannot start with a digit. Fix: `let age1 = 5; let age2 = 10;`

b) `let isPass = True;` → Invalid — JS is case-sensitive, keyword is lowercase `true`, not `True`.

c) `let isPass = 'true';` → this is a **string** "true", not a boolean.

18. Strings in JS

Strings are **text** or a sequence of characters (e.g. words, pages).

```
let name = "Tony Stark"; // double quotes
let role = 'ironman';    // single quotes
let char = 'a';
let num = '23';          // still a string!
let empty = " ";
```

Both `" "` and `' '` work for strings. Mixing quotes to include a quote character inside: `'this is "apple"'` works; but `'this is 'apple''` → **SyntaxError: Unexpected identifier** (mismatched quote nesting).

19. String Indices

Index = position of a character. JS strings use **0-based indexing**.

Char	T	O	N	Y	_	S	T	A	R	K
Index	0	1	2	3	4	5	6	7	8	9

```
let name = "TONY STARK";
name[0] // 'T'
name[1] // 'O'
name[1000] // undefined (out of range)
name.length // 10
typeof name.length // 'number'
name[name.length-1] // 'K' (last character)
```

20. Concatenation

Adding strings together using `+`.

```
"tony" + " " + "stark" → "tony stark"
"tony" + 1 → "tony1"

let firstName = "tony";
let lastName = "stark";
let fullName = firstName + lastName;
// "tonystark"
fullName = firstName + " " + lastName;
// "tony stark"
```

21. null & undefined in JS

undefined — A variable that has not been assigned a value is of type **undefined**.

```
let a;
a // undefined
typeof a // 'undefined'
```

null — represents the **intentional** absence of any object value. Must be explicitly assigned.

```
let a = null;
a // null
```

null is a reserved keyword — `let null = 4;` → **SyntaxError: Unexpected token 'null'**

22. Practice Questions

Declare your name as a string and print its length in JS.

Declare your first name as a string and print its first character.

Declare your first name as a string and print its last character.

What is output of: `"apnacollege" + 123` → `"apnacollege123"`
(number converted to string, concatenated)

Lengths: empty string `" "` → length **0**; string with single space `" "` → length **1**